

PROJECT REPORT

Implement Client Script & UI Policy (Incident) ServiceNow Developer Instance

Team ID: SWTID-2026-6763

Date: 03 October 2026

Maximum Marks: 10

TABLE OF CONTENTS

1. INTRODUCTION	3
1.1 Project Overview	3
1.2 Purpose	3
2. IDEATION PHASE	3
2.1 Problem Statement	3
2.2 Empathy Map Canvas	3
2.3 Brainstorming	3
3. REQUIREMENT ANALYSIS.....	4
3.1 Customer Journey map	4
3.2 Solution Requirement	4
3.3 Data Flow Diagram.....	5
3.4 Technology Stack.....	5
4. PROJECT DESIGN	5
4.1 Problem Solution Fit.....	5
4.2 Proposed Solution	6
4.3 Solution Architecture	6
5. PROJECT PLANNING & SCHEDULING.....	6
5.1 Project Planning	6
6. FUNCTIONAL AND PERFORMANCE TESTING.....	6
6.1 Performance Testing	6
7. RESULTS	7
7.1 Output Screenshots	7
8. ADVANTAGES & DISADVANTAGES	9
9. CONCLUSION.....	9
10. FUTURE SCOPE	9
11. APPENDIX.....	10

1. INTRODUCTION

1.1 Project Overview

This project implements Client Scripts and UI Policies on the ServiceNow Incident form. Client Scripts run in the user's browser to show guidance messages, react to field changes and validate data before an incident is submitted. UI Policies control how fields behave, making them mandatory, visible or read-only based on conditions such as State and Category. Together they help users enter complete and correct incident information without extra server-side processing.

1.2 Purpose

The purpose of this project is to improve the quality of incident data at the moment it is entered, to reduce rework caused by incomplete tickets, and to give users clear and immediate feedback on the Incident form.

2. IDEATION PHASE

2.1 Problem Statement

IT helpdesk teams often receive incidents with missing or poorly written details, for example a very short description, no configuration item for a network issue, or no close notes when an incident is resolved. Support staff then have to contact the requester again, which delays resolution. A form-level solution is needed that validates input and controls field behaviour as the user fills in the Incident form.

2.2 Empathy Map Canvas

Persona: IT Support Team Member (ServiceNow User)

THINK AND FEEL • Wants complete tickets the first time • Frustrated by repeated follow-ups • Values a clean, guided form	HEAR • Complaints about incomplete tickets • Feedback on data quality • Announcements about new form rules
SEE • Incidents with blank or vague fields • Agents chasing requesters for details • The standard Incident form	SAY AND DO • Reviews incoming incidents • Requests missing details from users • Fills in close information manually
PAIN • Incomplete incident data • Time lost on follow-ups • Inconsistent closure information	GAIN • Validated, complete incidents • Faster resolution • Consistent, guided data entry

2.3 Brainstorming

Problem statement: How might we make sure incident data is complete and valid at the time it is entered?

Idea	Description	Priority
onLoad Client Script	Show guidance when a new incident form opens	Medium
onChange Client Script	Alert the user when Priority is set to Critical	High
onSubmit Client Script	Block submission when the short description is too short	High

UI Policy: Resolved	Make Close notes and Close code mandatory when State is Resolved	High
UI Policy: Network	Make Configuration item mandatory for Network category	Medium
UI Policy: Closed	Make selected fields read-only once the incident is Closed	Medium

3. REQUIREMENT ANALYSIS

3.1 Customer Journey map

Stage	Open Form	Enter Details	Change Fields	Submit	Resolve	Close
Steps	Open new Incident form	Fill caller, category, description	Change Category / Priority	Click Submit	Set State to Resolved	Set State to Closed
Interactions	Incident form	Form fields	Client Script onChange	Client Script onSubmit	UI Policy	UI Policy
Goals	Start a new ticket easily	Describe the issue clearly	Get helpful feedback	Submit a valid ticket	Record the resolution	Close the ticket cleanly
Positive moments	Guidance message shown	Clear field labels	Instant alerts	Errors caught early	Mandatory fields guide user	Closed data protected
Negative moments	Unsure what to enter	Missing details	Unexpected value	Submission blocked	Extra fields required	Fields locked
Opportunities	Clearer instructions	Add field hints	More alert rules	Better error text	More conditions	Audit changes

3.2 Solution Requirement

FR No.	Functional Requirement (Epic)	Sub Requirement (Story / Sub-Task)
FR-1	Incident Form Access	Open the Incident form; view and enter incident details
FR-2	onLoad Client Script	Run on form load; show guidance for new incidents
FR-3	onChange Client Script	React to field changes such as Priority and Category
FR-4	onSubmit Client Script	Validate short description; block invalid submission
FR-5	UI Policy - Mandatory	Make Close notes / Close code mandatory when Resolved
FR-6	UI Policy - Conditional	Make Configuration item mandatory for Network; read-only when Closed
FR-7	Messages and Error Handling	Show clear information and error messages to the user
FR-8	Testing and Validation	Test each script and policy; verify results

3.3 Data Flow Diagram

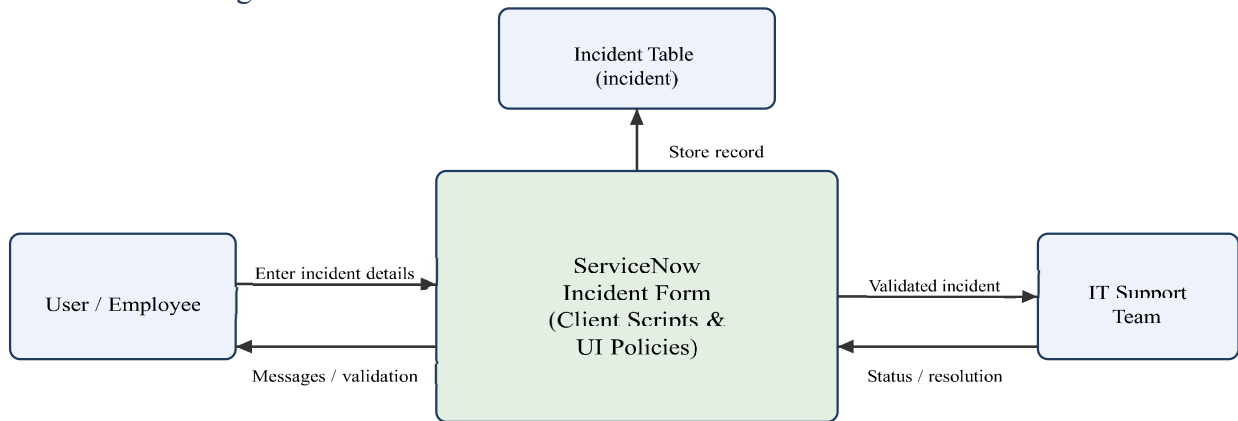


Figure: DFD Level 0 - Client Script & UI Policy on the Incident form

3.4 Technology Stack

Component	Technology
Platform	ServiceNow Developer Instance
Client-side logic	Client Scripts (JavaScript, g_form API)
Form behaviour	UI Policies (conditions and actions)
Module	Incident Management / System Definition
Environment	ServiceNow Development Environment

4. PROJECT DESIGN

4.1 Problem Solution Fit

S.No.	Parameter	Description
1	Problem Statement	IT helpdesks receive incidents with missing or invalid details, causing delays and repeated follow-ups with requesters.
2	Idea / Solution description	Use Client Scripts to validate and guide users on the Incident form, and UI Policies to make fields mandatory, visible or read-only based on conditions.
3	Novelty / Uniqueness	Enforces data quality instantly on the client side using native ServiceNow features, with no external tools and little coding.
4	Social Impact / Customer Satisfaction	Users get immediate guidance and support staff receive complete tickets, reducing frustration and speeding up resolution.
5	Business Model (Revenue Model)	Internal process improvement that reduces rework and administrative effort for IT support.

6	Scalability of the Solution	Scripts and policies can be extended to other tables, fields and conditions, or reused across instances using update sets.
---	-----------------------------	--

Implement Client Script & UI Policy (Incident)

4.2 Proposed Solution

When a user opens the Incident form, an onLoad Client Script displays guidance. When fields such as Priority change, onChange Client Scripts react immediately. When the form is submitted, an onSubmit Client Script validates the short description and stops invalid submissions. UI Policies evaluate conditions on State and Category and apply actions that make fields mandatory, visible or read-only. The combined behaviour ensures complete, valid incidents.

4.3 Solution Architecture

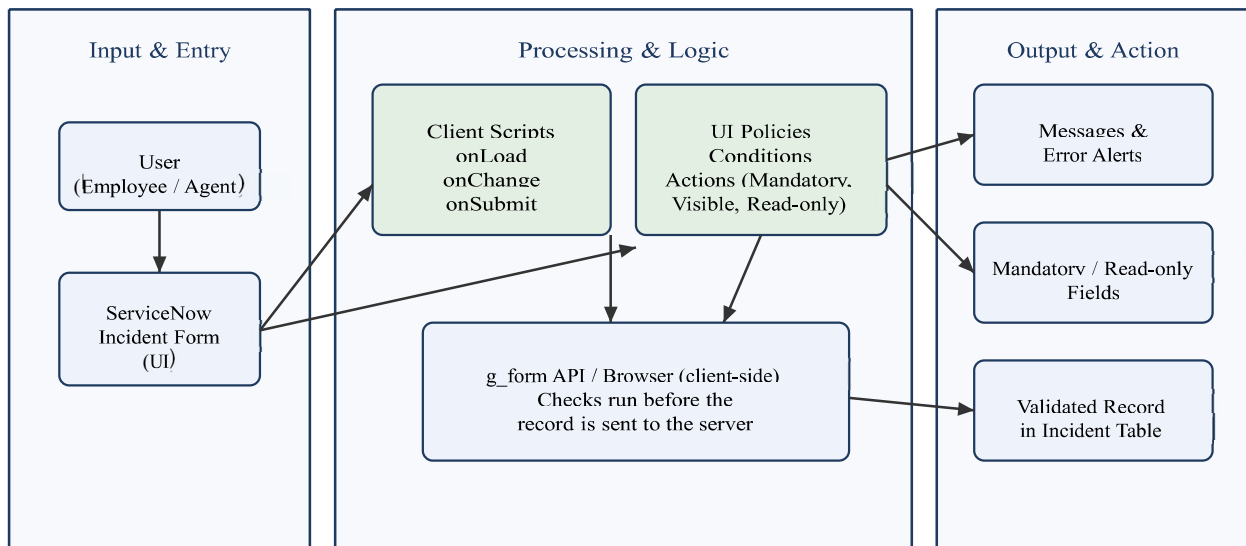


Figure: Solution Architecture

5. PROJECT PLANNING & SCHEDULING

5.1 Project Planning

Sprint	Tasks	Duration	Dates	Status
Sprint-1	8 (requirements, instance setup, onLoad / onChange / onSubmit Client Scripts)	6 Days	22 Sep - 27 Sep 2026	Completed
Sprint-2	9 (UI Policies, testing, UAT, documentation, GitHub upload)	6 Days	28 Sep - 03 Oct 2026	Completed

6. FUNCTIONAL AND PERFORMANCE TESTING

6.1 Performance Testing

Five functional and three performance test cases were executed. All 8 passed with no failures (100% pass rate).

Test ID	Scenario	Expected Result	Actual Result	Result
FT-01	Client Script - onLoad Message	Guidance message shown on new form	Message displayed correctly	PASS

FT-02	Client Script - onChange (Priority)	Critical alert shown for Priority 1	Alert displayed	PASS
-------	-------------------------------------	-------------------------------------	-----------------	------

Implement Client Script & UI Policy (Incident)

FT-03	Client Script - onSubmit Validation	Short descriptions blocked with error	Invalid input rejected; valid saved	PASS
FT-04	UI Policy - Mandatory Field	Close notes / code mandatory when Resolved	Fields mandatory; save blocked if empty	PASS
FT-05	UI Policy - Conditional Behaviour	CI mandatory for Network; read-only when Closed	Worked as configured	PASS
PT-01	Form Load Time Test	Form loads within acceptable time	No noticeable delay	PASS
PT-02	Client Script Response Time	Scripts respond instantly	Instant response	PASS
PT-03	Repeated Execution Reliability	Consistent behaviour every time	Consistent across incidents	PASS

Testing Type	Test Cases	Passed	Failed
Functional Testing	5	5	0
Performance Testing	3	3	0
Total	8	8	0

7. RESULTS

7.1 Output Screenshots

The screenshot shows the ServiceNow 'Incident > Create New' form. At the top, a message bar states: 'INFO: Incident Priority has been dynamically set to '1 - Critical' based on Urgency.' The form fields are as follows:

- Number:** INC10012
- Urgency:** 1 - High (dropdown menu)
- Priority:** 1 - Critical (dropdown menu)
- Short Description:** (text input field)
- Configuration Item:** (text input field with a search icon)
- Description:** (large text area)
- Customization Test:** (checkbox, currently unchecked)

At the bottom of the form, there are buttons for '< UI Actions', 'Submit' (highlighted in blue), and 'Resolve'. A 'Resolve All' button is also visible in the top right corner of the form area.

Fig 1: Incident form with onLoad information message

Incident - Create New

servicenow.com/ak/arssv:rent1/locs/Incident>CreateNew

servicnom

Incident > Create New

INFO: Please enter complete incident details.

Number: INC10011

Urgency: 1 - High

Priority: 1 - Critical

Short Description

Configuration Item

Configuration Item

End Actors

Description

Submit Resolve

Fig 2: Critical priority alert (onChange Client Script)

Incident - Create

servicenow.com/aardurk-reporting/Incident/Incident/INC10013>createnewNew

servicenow

Incident > Create New

Error: The following mandatory fields are not filled: Resolution code, Resolution notes.

Create New

Number: INC10013

Urgency: 1 - High

Urgency: 1 - High

Priority: 1 - Critical

Mode

States: None

Description

* Close code

Priority: 1 - Critical

State: Resolved

State: Resolved

Aolient

Close notes

Resolution Information

* Resolution code

* Resolution notes

* Close code

* Close notes

Submit Resolve

Fig 3: UI Policy - mandatory Close notes / Close code

8.ADVANTAGES & DISADVANTAGES

Advantages

- Gives users instant feedback without a server round trip.
- Improves the completeness and quality of incident data.
- Uses native ServiceNow features, so rules are easy to maintain.
- Reduces follow-ups and rework for support teams.

Disadvantages

- Client Scripts run only in the browser, so data entered through other channels (such as imports or APIs) is not validated.
- Too many scripts and policies on one form can affect load time and become hard to manage.
- Rules are fixed and must be updated manually when requirements change.

9.CONCLUSION

Project Summary

The project implements Client Scripts and UI Policies on the ServiceNow Incident form to guide users and enforce data quality. All functional and performance tests passed, confirming that the scripts and policies behave as designed.

Key Outcomes

- Client Scripts display messages and validate input on the Incident form.
- UI Policies control mandatory, visible and read-only fields based on conditions.
- The form performed reliably and consistently during testing.
- All 8 test cases passed (100%).

10.FUTURE SCOPE

Enhancement Opportunities

- Add more validation rules and conditions for other fields.
- Add server-side Business Rules to enforce the same checks for all channels.
- Apply the approach to other tables such as Change and Problem.

Future Automation

- Combine with Flow Designer to automate assignment and notifications.
- Use Data Policies for validation that applies to every channel.
- Add dashboards to track data-quality trends.

11.APPENDIX

Source Code (if any)

```
// onLoad - Incident function
onLoad() { if
(g_form.isNewRecord())
    g_form.addInfoMessage('Please enter complete incident details. ');
}
// onChange - Priority
function onChange(control, oldValue, newValue, isLoading) { if
(isLoading || newValue === '') return; if (newValue == '1')
    g_form.addInfoMessage('Critical incident: needs immediate attention. '); }
// onSubmit - Incident function
onSubmit() {
    if (g_form.getValue('short_description').length < 10) {
        g_form.addErrorMessage('Short description must be at least 10 characters. '); return false;
    }
}
```

Dataset Link

Not applicable. No external dataset is used; the project works on the ServiceNow Incident table.

GitHub & Project Demo Link

GitHub: <https://github.com/srinandhabala0666/Implement-Client-Script-UI-Policy-Incident->

Project Demo:

<https://drive.google.com/drive/u/0/mobile/folders/1tJEdhbbE4r9IXNUVkJbTURAmwLSUoRA?usp=sharing>